

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

Pseudocode (Multithreaded Producer–Consumer Design):

This version separates capture, processing, and display into concurrent threads connected by bounded queues, which minimizes perceived delay by never blocking frame capture on recognition work.

```
MODULE FaceSecuritySystem
```

```
// ===== GLOBAL STRUCTURES =====
```

```
STRUCT KnownFace { label, feature_vector }
```

```
GLOBAL known_faces_db : LIST of KnownFace
```

```
GLOBAL frame_queue : THREAD-SAFE QUEUE (max_size = 5)
```

```
GLOBAL result_queue : THREAD-SAFE QUEUE (max_size = 5)
```

```
GLOBAL running_flag = TRUE
```

```
// ===== FUNCTION: Load Known Dataset =====
```

```
FUNCTION BuildDatabase(dataset_path)
```

```
    known_faces_db = EMPTY LIST
```

```
    FOR EACH person_folder IN dataset_path DO
```

```
        FOR EACH image_file IN person_folder DO
```

```
            img = READ_IMAGE(image_file)
```

```
            face = DETECT_SINGLE_FACE(img)
```

```
            vec = COMPUTE_EMBEDDING(face)
```

```

        APPEND KnownFace(person_folder.name, vec) TO known_faces_db
    END FOR
END FOR
RETURN known_faces_db
END FUNCTION

// ===== THREAD 1: Capture Producer =====
FUNCTION CaptureThread(camera_source)
    capture = OPEN_CAMERA(camera_source)
    SET_BUFFER_SIZE(capture, 1)    // always keep the freshest frame
    WHILE running_flag DO
        ok, frame = capture.READ()
        IF NOT ok THEN CONTINUE
        IF frame_queue.IS_FULL() THEN
            frame_queue.DROP_OLDEST() // avoid latency buildup
        END IF
        frame_queue.PUSH(frame)
    END WHILE
    capture.RELEASE()
END FUNCTION

// ===== THREAD 2: Detection + Recognition Worker =====
FUNCTION ProcessingThread()
    WHILE running_flag DO
        IF frame_queue.IS_EMPTY() THEN
            SLEEP(1 ms)
            CONTINUE
        END IF

        frame = frame_queue.POP()

        // --- Preprocess ---
        small = DOWNSCALE(frame, factor = 0.5)    // cheaper detection
        gray = TO_GRAYSCALE(small)
        gray = NORMALIZE_ILLUMINATION(gray)

        // --- Detect all faces in parallel-friendly batch ---
        boxes = RUN_FACE_DETECTOR(gray)           // returns list of (x,y,w,h)
    
```

```

SCALE_BOXES_BACK(boxes, factor = 0.5)      // map back to original frame size

annotations = EMPTY LIST

// --- Recognize each detected face ---
FOR EACH box IN boxes DO
    crop = CROP(frame, box)
    vec = COMPUTE_EMBEDDING(crop)

    label, confidence = MATCH_AGAINST_DATABASE(vec, known_faces_db)
    APPEND (box, label, confidence) TO annotations
END FOR

IF result_queue.IS_FULL() THEN
    result_queue.DROP_OLDEST()
END IF
result_queue.PUSH( (frame, annotations) )
END WHILE
END FUNCTION

FUNCTION MATCH_AGAINST_DATABASE(vec, db)
    closest_label = "Unknown" ; closest_dist = INFINITY
    FOR EACH entry IN db DO
        d = EUCLIDEAN_DISTANCE(vec, entry.feature_vector)
        IF d < closest_dist THEN
            closest_dist = d ; closest_label = entry.label
        END IF
    END FOR
    IF closest_dist > THRESHOLD THEN RETURN ("Unknown", closest_dist)
    RETURN (closest_label, closest_dist)
END FUNCTION

// ===== THREAD 3 (MAIN): Display / UI Loop =====
FUNCTION DisplayLoop()
    WHILE running_flag DO
        IF result_queue.IS_EMPTY() THEN
            key = POLL_KEY(timeout = 1 ms)
        ELSE

```

```

(frame, annotations) = result_queue.POP()

FOR EACH (box, label, confidence) IN annotations DO
    (x, y, w, h) = box
    colour = (label == "Unknown") ? RED : GREEN
    DRAW_RECT(frame, x, y, w, h, colour)
    DRAW_TEXT(frame, label + " (" + ROUND(confidence,2) + ")", (x, y-10))
END FOR

DRAW_TEXT(frame, "Faces: " + LENGTH(annotations), (10, 30))
SHOW(frame, window = "Face Security Monitor")
key = POLL_KEY(timeout = 1 ms)
END IF

IF key == ESC OR key == 'q' THEN
    running_flag = FALSE
END IF
END WHILE
DESTROY_ALL_WINDOWS()
END FUNCTION

// ===== ENTRY POINT =====
FUNCTION Main()
    BuildDatabase("registered_persons/")
    START_THREAD(CaptureThread, args = camera_source)
    START_THREAD(ProcessingThread)
    DisplayLoop()      // runs on main thread, keeps UI responsive
    JOIN all threads
END FUNCTION

CALL Main()

END MODULE

```

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

Pseudocode (Object-Oriented Design with Kalman-Filter Tracking):

This version models each vehicle as a VehicleTrack object with its own Kalman filter for motion prediction, uses lightweight background-subtraction contours for detection, and determines crossing via a side-of-line sign change so each track can only be counted once.

```
MODULE TrafficCountingSystem
```

```
// ===== CLASS DEFINITION =====
```

```
CLASS VehicleTrack
```

```
  FIELDS: id, kalman_filter, last_box, age, missed_frames, counted (bool), side_of_line (prev)
```

```
  METHOD Predict()
```

```
    predicted_center = kalman_filter.PREDICT()
```

```
    RETURN predicted_center
```

```
  END METHOD
```

```
  METHOD Update(new_box)
```

```
    center = CENTER_OF(new_box)
```

```
    kalman_filter.CORRECT(center)
```

```
    last_box = new_box
```

```
    missed_frames = 0
```

```
    age = age + 1
```

```
  END METHOD
```

```
END CLASS
```

```

// ===== GLOBAL STATE =====
GLOBAL active_tracks : LIST of VehicleTrack = EMPTY
GLOBAL next_id = 1
GLOBAL total_count = 0
GLOBAL LINE_A, LINE_B           // endpoints of virtual counting line
GLOBAL ROI_POLYGON              // detection region
CONSTANT MAX_MISSED = 10
CONSTANT MATCH_DIST_THRESHOLD = 60    // pixels

// ===== HELPER FUNCTIONS =====
FUNCTION SideOfLine(point, A, B)
    // returns +1, -1, or 0 depending on which side of line AB the point lies
    cross = (B.x - A.x)*(point.y - A.y) - (B.y - A.y)*(point.x - A.x)
    IF cross > 0 THEN RETURN 1
    ELSE IF cross < 0 THEN RETURN -1
    ELSE RETURN 0
END FUNCTION

FUNCTION GreedyMatch(tracks, detections)
    // Pairs each track's predicted position to the nearest unclaimed detection
    pairs = EMPTY LIST
    unmatched_dets = COPY(detections)
    unmatched_tracks = COPY(tracks)

    FOR EACH track IN tracks (sorted by age DESC) DO
        predicted = track.Predict()
        best_det = NULL ; best_dist = MATCH_DIST_THRESHOLD
        FOR EACH det IN unmatched_dets DO
            d = EUCLIDEAN_DISTANCE(predicted, CENTER_OF(det))
            IF d < best_dist THEN
                best_dist = d ; best_det = det
            END IF
        END FOR
        IF best_det != NULL THEN
            APPEND (track, best_det) TO pairs
            REMOVE best_det FROM unmatched_dets
            REMOVE track FROM unmatched_tracks
        END IF
    END FOR

```

```

END FOR
RETURN (pairs, unmatched_dets, unmatched_tracks)
END FUNCTION

// ===== MAIN PIPELINE =====

FUNCTION Main(video_source)
    stream = OPEN_VIDEO(video_source)
    IF NOT stream.IS_OPENED() THEN
        PRINT "Unable to open traffic feed" ; RETURN
    END IF

    background_model = INIT_BACKGROUND_SUBTRACTOR(history = 500, varThreshold = 40)

    LOOP
        ok, frame = stream.READ()
        IF NOT ok THEN BREAK          // end of stream — terminate cleanly

        // ---- Preprocessing ----
        masked = APPLY_ROI_MASK(frame, ROI_POLYGON)
        blurred = GAUSSIAN_BLUR(masked, (5,5))
        fg_mask = background_model.APPLY(blurred)
        fg_mask = MORPH_OPEN(fg_mask) ; fg_mask = MORPH_DILATE(fg_mask)

        // ---- Detection via contours (lightweight alternative to a DNN) ----
        contours = FIND_CONTOURS(fg_mask)
        detections = EMPTY LIST
        FOR EACH c IN contours DO
            IF CONTOUR_AREA(c) > MIN_VEHICLE_AREA THEN
                box = BOUNDING_RECT(c)
                APPEND box TO detections
            END IF
        END FOR

        // ---- Association: predicted track positions vs new detections ----
        (matched, new_dets, lost_tracks) = GreedyMatch(active_tracks, detections)

        FOR EACH (track, det) IN matched DO
            track.Update(det)

```

```

END FOR

FOR EACH det IN new_dets DO
    new_track = NEW VehicleTrack(id = next_id, box = det)
    next_id = next_id + 1
    APPEND new_track TO active_tracks
END FOR

FOR EACH track IN lost_tracks DO
    track.missed_frames = track.missed_frames + 1
END FOR

REMOVE FROM active_tracks WHERE missed_frames > MAX_MISSED

// ---- Line-crossing counting logic (single-count guarantee) ----
FOR EACH track IN active_tracks DO
    current_side = SideOfLine(CENTER_OF(track.last_box), LINE_A, LINE_B)
    IF track.side_of_line IS DEFINED
        AND track.side_of_line != current_side
        AND NOT track.counted THEN
            total_count = total_count + 1
            track.counted = TRUE      // permanently flag — no recount possible
        END IF
        track.side_of_line = current_side
    END FOR

// ---- Visualization ----
DRAW_LINE(frame, LINE_A, LINE_B, RED, 2)
FOR EACH track IN active_tracks DO
    DRAW_RECT(frame, track.last_box, GREEN)
    DRAW_TEXT(frame, "ID " + track.id, TOP_LEFT(track.last_box))
END FOR
DRAW_TEXT(frame, "Count: " + total_count, (10, 30))
SHOW(frame, window = "Traffic Counter")

// ---- Efficient termination check ----
IF POLL_KEY(1 ms) == 'q' THEN BREAK
END LOOP

```



```
stream.RELEASE()
DESTROY_ALL_WINDOWS()
PRINT "Session ended. Total vehicles counted: " + total_count
END FUNCTION

CALL Main("traffic_feed.mp4")

END MODULE
```